

Protocole de test manuel — digest d'actions employeur (employer_actions)

Objectif

Vérifier de bout en bout le fonctionnement des mails de “digest” envoyés aux employeurs pour les notifier des actions en attente sur leurs candidatures, leurs conventions et leurs offres de stage.

Le digest regroupe des MailActionItem par niveau d'urgence (low, medium, high, critical) et envoie un mail récapitulatif par niveau, via `Services::EmployerActions::EmployerDigestMailer`.

Pré-requis

- Un compte employeur de test avec une offre publiée et des semaines disponibles.
- Un ou plusieurs comptes élèves de test.
- En environnement review, les mails sont envoyés immédiatement (`deliver_now`). En local, vérifier que Sidekiq tourne, ou déclencher manuellement les tâches rake :

```
bin/rails digest_mailers:send_low_urgency_emails
bin/rails digest_mailers:send_medium_urgency_emails
bin/rails digest_mailers:send_high_urgency_emails
bin/rails digest_mailers:send_critical_urgency_emails
```

- Avoir accès à la boîte mail de l'employeur (ou Letter Opener / Mailcatcher en local).
- Pouvoir inspecter la table `mail_action_items` (console Rails ou DB) pour vérifier les champs `action_name`, `action_type`, `urgency_level`, `resolved_at`, `deliveries_count`, `last_notified_at`, `stale_at`.

Principe général à vérifier pour chaque scénario

Pour chaque action testée : 1. **Déclenchement** : l'action métier (candidature, convention, offre) crée bien un MailActionItem avec le bon `action_name`, `action_type` et `urgency_level` (voir tableau de référence ci-dessous), avec `resolved_at`: nil. 2.

Résolution : si l'état change de façon à rendre la notification obsolète (ex: candidature à nouveau soumise, convention signée, etc.), l'item doit être marqué résolu (`resolved_at` renseigné) et **ne pas** générer de mail — sans pour autant supprimer ou résoudre par erreur d'autres items en attente liés au même enregistrement (cf. scénario de non-régression plus bas). 3. **Envoi** : si l'item reste en attente (`pending`, non périmé `stale_at`), le mail de digest correspondant au niveau d'urgence doit être envoyé à l'employeur, et l'item doit

voir son `deliveries_count` incrémenté et `last_notified_at` renseigné. 4. **Non-renvoi après résolution** : un item résolu ou ayant atteint son `max_deliveries_count` ne doit plus générer d'envoi, et finit par être supprimé (purge par le Resolver).

Tableau de référence des actions

action_name	action_type
new_internship_application	pending_internship_application
canceled_internship_application_by_student	pending_internship_application
restored_internship_application	pending_internship_application
cancel_by_student_confirmation	pending_internship_application
candidate_chose_another_internship	pending_internship_application
candidate_restored_by_student	pending_internship_application
canceled_internship_application	pending_internship_application
internship_application_transferred	pending_internship_application
new_agreement_to_fill_in	pending_internship_agreement
agreement_signed_by_another	pending_internship_agreement
agreement_to_sign	pending_internship_agreement
signatures_enabled	pending_internship_agreement
agreement_signed_by_all	pending_internship_agreement
internship_offer_unpublished	pending_internship_offer
internship_offer_removed	pending_internship_offer

Scénarios à dérouler

1. Nouvelle candidature → mail “nouvelle candidature”

- L’élève postule. Vérifier la création du MailActionItem `new_internship_application` (medium), puis l’envoi du digest medium et la réception du mail par l’employeur.

2. Candidature lue puis annulée par l’élève → mail d’annulation

- L’employeur ouvre/lit la candidature (`read_by_employer`), puis l’élève l’annule.
- Vérifier la création de `canceled_internship_application_by_student` (medium), son maintien en attente (`resolved_at: nil`), et la réception du mail au prochain digest medium.
- **Non-régression** : vérifier que le mail “nouvelle candidature” associé à cette même candidature (résolu entre-temps car la candidature n’est plus submitted) ne fait pas disparaître/résoudre par erreur le mail d’annulation — les deux items doivent être traités indépendamment.

3. Candidature jamais lue puis annulée par l’élève → pas de mail d’annulation

- L’élève postule, l’employeur **ne consulte pas** la candidature, l’élève annule.
- Vérifier que `canceled_internship_application_by_student` est créé puis marqué `resolved_at` (résolu) avant tout envoi, et qu’**aucun** mail d’annulation n’est reçu par l’employeur.

4. Candidature restaurée après annulation

- Annuler puis restaurer une candidature. Vérifier la création de `restored_internship_application` et son comportement de résolution selon que la candidature a été lue ou non par l’employeur (même logique que le scénario 2/3).

5. Confirmation d’annulation par l’élève

- Dérouler le flux de confirmation d’annulation (`cancel_by_student_confirmation`, urgency high). Vérifier création, envoi au digest high/critical, et résolution si la candidature change d’état avant l’envoi.

6. Convention : à compléter, à signer, signée par tous

- Créer une convention (`new_agreement_to_fill_in`), la faire signer partiellement (`agreement_to_sign`, `agreement_signed_by_another`), puis entièrement (`agreement_signed_by_all`). Vérifier la création et la résolution successive des MailActionItem correspondants, et la réception des mails de digest associés (urgency medium).

7. Offre dépubliée / supprimée

- Dépublier une offre : vérifier `internship_offer_unpublished` (low) et le mail reçu au digest low.

- Supprimer une offre : vérifier `internship_offer_removed` (high) et le mail reçu au digest high/critical.

8. Péremption (`stale_at`) et plafond d'envoi (`max_deliveries_count`)

- Provoquer une notification dont le `stale_at` est dans le passé (ou attendre qu'il le devienne) : vérifier qu'elle est purgée par le `Resolver` sans envoi de mail.
- Provoquer plusieurs cycles de digest sur une même notification non résolue : vérifier qu'au-delà de `max_deliveries_count` envois, l'item est purgé et plus aucun mail n'est envoyé pour cette action.

9. Regroupement multi-niveaux dans un même digest

- Avoir simultanément des `MailActionItem` en attente à plusieurs niveaux d'urgence pour un même employeur (ex: low + medium + high). Déclencher le digest d'un niveau donné et vérifier qu'il regroupe bien les niveaux égaux ou supérieurs (cf. `urgency_levels_sum_up`), sans dupliquer les actions déjà couvertes par un digest de niveau inférieur déjà envoyé.

Points d'attention transverses

- Un employeur sans actions en attente ne doit recevoir aucun mail de digest (cf. log "No pending and not overdue actions to notify").
- Les actions résolues doivent disparaître de la base après le passage du `Resolver` (purge), sans avoir généré de mail.
- Vérifier qu'aucune action n'est "perdue" (résolue par erreur) ni "dupliquée" (envoyée plusieurs fois au-delà de son plafond) lors de l'enchaînement de plusieurs scénarios sur la même candidature/convention/offre.